02562 Exam Project - Procedural solid texturing

s214643

1 INTRODUCTION

In the present past, animation and computer graphics have become ever more relevant an popular subjects. Every child have seen at least a dozen of Disney's animation movies. Teenagers all over the world have played the latest in graphical video games. Each media ever pushing the boundaries of what is possible on the digital screen. Each making the virtual worlds ever more realistic and fantastic. But pushing the realism comes at a cost, each element of the graphics have to be ever more detailed to fool the eye. Each wall, each rock, each trinket has to be created using realistic textures and models. Overlaying each object with real-life scans or photos of actual objects would cause an abundance of issues, both in loading time and permutations. Instead we must define textures using vector representations and mathematical expressions, too make easily accessible textures for any object.

In this project I will go through the basics of procedural textures and how to make them. For this I will be using the tool; Blender, and a learning by doing strategy to implement various shader techniques, showcasing simple implementations.

2 METHOD

Creating procedural textures in Blender is a creative process. There is no how-to or factual reasoning that will get you there. Instead one has to rely on the artistic sense and mix it with a mathematical understanding. As I'm an engineer and not an artist I dare not try to explain how the creative process work, instead I will explain the mathematical understanding needed for generating procedural textures. There are a couple of functions (that are pre-implemented in Blender), which one has to understanding. Starting off with the random generation.

2.1 Perlin Noise

As the main focus of the project is on the implementation nature-like textures, randomness is rather important. Randomness allows us to make textures less robotic and generated. It helps give textures a more "non-human" look, which is essential for realism. In Blender the most common noise is Perlin noise. The method was originally introduced by Ken Perlin in 1982 as he was "frustrated" with the machine-like texture of CGI (wikipedia, 2025).

The noise texture is created in three steps:

- First a grid is placed over object for which ones whishes to create a random texture. In the case of this report, the grid is tree-dimensional, but it could be any number of dimensions. When the grid is placed, a random vector is assigned to each grid intersection.

- Secondly, to find the value of a given pixel in the object we first find the 2^n (where n is the number of dimensions) grid intersections closest to the point (for a 3d object we are looking for 8 intersections). Now, calculate an offset vector between each intersection and the given pixel. Then, to find the candidate value of that pixel, we find the dot product between the random vector and the offset vector.

- The last step is an interpolation between the each of the 2^n (in this case 8) points. The interpolation is typically performed with a function, that passes gives an output in the range of $[-1; 1]$. Depending on the interpolation function the output can either be in 3-channels, RGB, or a scalar value, black and white.

Later in this project I will also refer to fractal Brownian motion (fBm) noise which essentially is Perlin noise stacked and added together in an iterative process (Patricio Gonzalez Vivo, 2015).

2.2 Voronoi texture

Voronoi texture is another random function which allows us to create natural-looking color schemes on the objects. For the Voronoi texture, we assign a set of random dots spanning the entire object (however many dimensions that may be). Then a function is assigned which maps a value to points based on how far away from the random dots they are.

2.3 Color Ramp

An important feature in the Blender shader editor is the color ramp. This tool allows you to map a gradient to some scalar factor. The gradient can have any number of vertices, but in this project only two are used (one on either side). Essentially this node creates a linear interpolation between the defined vertices, calculating a normalized value based on the input-factor. Mathematically it is defined as:

$$C_{\text{out}} = (1 - g)C_0 + gC_1$$

Where C_0 and C_1 are the two colors in the color ramp and g is the input factor. It is most commonly known to create gradient colors, but can also be used to distort surfaces of objects, as we will see later in the project.

3 IMPLEMENTATION

This project will feature three different procedural textures; marble, wood and rocky lava. Each adding a new level of complexity to the process. For the full setup, see the blender file textures: `./project/project.blend`.

3.1 Marble

The marble texture is the most simple of the three. It is mainly focused on a noisy texture with the right values. Since there is no color in the marble texture it is possible to create only using scalar values. On the other hand the texture relies heavily on some of the aforementioned build-in Blender texture functions. The equation for this texture is:

$$C_{\text{marble}} = 0.4 \cdot \text{Ramp}(\text{Voronoi}(\text{fBm}(\mathbf{p_{x,y,z}}))) + 0.6 \cdot \text{Ramp}(\text{Voronoi}(\text{fBm}(\mathbf{p_{x,y,z}})))$$

Unpacking this equation is rather simple. The variable; $\mathbf{p_{x,y,z}}$ is the normalized location of a pixel on the object. This position is first subjected to an fBm noise function, and later the Voronoi random texture. From here two different color ramps are used, one for each part. The color ramps as strictly black and white and only used to create the different tones and sharper lines in the marble texture. Finally a mix color node is used to mix the output of each ramp with a 40/60 split. In layman's terms the marble texture is created by making a number of "cells" with the Voronoi texture, but distorting the input with the fBm noise. The lines of the "cells" are later sharpened in two different levels using the color ramps. This, fairly simple, technique creates a pseudo-realistic marble texture with clear, crackling, lines and some natural discoloration.

3.2 Wood

The wooden texture is a bit more complicated. This involves some more serious mathematical operations and is relying more heavily on following natures patterns.

The equation for the wooden texture is as follows:

$$C_{\text{wood}} = \text{CR}(\sin(||\mathbf{p_{x,y}} \cdot \mathbf{t_{x,y}}|| + \text{fBm}(\mathbf{p_{x,y,z}}))) + \text{CR}(\text{fBm}(\mathbf{p_{x,y,z}} \cdot \mathbf{t_{x,y,z}}))$$

Now, let's break this down for a better understanding. We will start out with the part creating the rings of the wood: $||\mathbf{p_{x,y}} \cdot \mathbf{t_{x,y}}||$. We use $p_{x,y}$ to describe a normalized position of a given pixel in the objects bounding box. The other component, $t_{x,y}$ is a translation, that allows us to move, and possibly, stretch the texture on the object. Calculating the norm of that sum gives us a 2D parabola shape with a minimum in the center of the object. By applying a sine function to that, we get the ring structure typical for wood. This is the same function used as when running a sinusoidal kernel over a 2nd degree polynomial. Then what is the purpose of the fBm function? Once again we need to add a certain level of randomness for the pattern to look natural. This allows us to simulate a more realistic wood ring structure that does not follow a perfect mathematical pattern.

The second part of the equation: $\text{fBm}(\mathbf{p_{x,y,z}} \cdot \mathbf{t_{x,y,z}})$ adds a grain to the wood texture. Similar to the ring structure, we use the coordinates of the pixel in the bounding box and a translation to sample from the fBm noise. It is important to note that the translation in this case is used to stretch the texture along the x and y axis, while leaving the z axis untouched. This is done to ensure that the grain follows a clear direction as wood grains do.

In both cases the color ramp (CR()), used as the final layer, are set to grade between a light and dark brown color, simulating the natural color of wood.

3.3 Lava Rock

Now to the last, but definitely not the least, the lava rock. This texture adds yet another layer of complexity to the bunch. Not only are we alternating the colors on the surface layer of the object, we are now also alternating the surface area creating the spiky rocky texture, and we are adding an emission layer to create that smooth glow of lava. As this design is more complex it is made with heavy inspiration by Blender artist "Ryan King Art" on Youtube (King, 2023).

Now, moving on to the texture, there are three parts to this texture; the rock texture, the lava texture, the surface displacement displacement.

3.3.1 Rock Texture

This part is quite simple and mostly involves parts we have seen before. It can be Mathematically described as:

$$C_{\text{rock}} = \text{CR}(\text{fBm}(\mathbf{p_{x,y,z}}))$$

Which simply defines a color ramp for some randomly distorted object coordinates. The color ramp grades between black and a dark brown color to simulate a brunt rocky texture. Now, there is one more part to this. Apart from alternating the color, this part also slightly alternates the normal of the texture, creating this rough, bumpy surface. This is simply done by letting the randomly distorted coordinates influence the normal of the BSDF shader using a Blender "bump-node".

3.3.2 Lava Texture

The lava texture creates the color scheme and texturing for the lava flow in the texture. It was important for the the lava to not only create the flowy hot liquid, but also create a "sort-of" glow

to the texture. In pure math, this texture is very similar to the rock texture, only here we are alternating the emission layer, instead of the base color of the BSDF node:

$$E_{\text{lava}} = \text{CR}(\text{fBm}(\mathbf{p_{x,y,z}}))$$

Where E describes an emission. In this case the color ramp is obviously set differently than for the rock texture. Namely, here we use a gradient between a hot yellow/orange and a red color. Again we are using the randomly distorted object coordinates to alternate the normal of the texture to create a slightly more rough look for the lava as it is a quite viscous liquid.

3.3.3 Surface Displacement

Firstly we create a weakly distorted set of object coordinates, which are run through a Voronoi texture, to create "cells".

$$\text{Cell}_{x,y,z} = \text{Voronoi}(0.3\text{fBm}(\mathbf{p_{x,y,z}}) + 0.7\mathbf{p_{x,y,z}})$$

As the avid reader may notice, this is rather similar to the marble texture from earlier. It creates a black-and-white look with natural looking areas. In the lava rock texture it will be used for two things; displacing the surface area of the texture and alternating between the rock and lava textures. Again we are using the color ramp to create the gradient, only this time it is not used as a color gradient but rather a gradient between different textures and displacements, we will use DF as the gradient Voronoi cells: $\text{DF}_{x,y,z} = \text{CR}(\text{Cell}_{x,y,z})$. Essentially, we are creating a function which will make the extruding factors in the texture us the rocky look and the intruding factors use the lava texture.

With this we can write the final mathematical term for texturing the rocky lava:

$$C_{\text{rocky lava}} = (1 - \text{DF}_{x,y,z})E_{\text{lava}} + \text{DF}_{x,y,z}C_{\text{rock}}$$

And the displacement as:

$$D_{\text{rocky lava}} = \text{DF}_{x,y,z}$$

Where D is the object displacement mapped with the displacement node in Blender.

4 RESULTS

While the mathematical notations for the textures are pretty in themselves we need to have a look at the final renders to evaluate how well they fit the natural elements they claim to be. For each render there are two objects, the Stanford bunny used as a reference marker, and a texture-specific object which present the texture in the best way.

4.1 Marble

For the marble texture the special object is a set of marble plates/tiles. The result of the marble texture can be seen in Figure 1. Marble comes in many forms and types, which may ultimately be what saves this texture. Are you generally in doubt about what you are supposed to see on the screen? Probably not, but it are you certain that this is real marble? Also no. It looks plenty real to pass, and especially if seen among other rendered textures like it would in an animation movie or a video game. Real marble would presumably have a more washed out texture which leans more to a grey color, rather and the pure white and pure black texture seen in this render.

Figure 1. Marble texture render with Stanford bunny and tiles, black background. Rendered until convergence in a 1080x1080 frame. See `./project/renders/marble_1024.png` for clearer version.

4.2 Wood

For the wood texture there is a bit more attention to detail for the natural structure of the material. Here the special object is a set of wooden planks. The result of this render can be seen in page 6. Here the wood texture actually looks believable, apart from the fact that they may be cut a little too clean i.e. lacking some displacement. The wood rings follow a natural-looking pattern, which also works well on the flat surface of the plank. What could be lacking is some knots in the wood, which often occur where branches have been on the tree, but since these are not always visible on processed wood it is a forgivable omission. Looking at the reference bunny, it is a bit more clear

that the texture is rendered as a wood worker probably wouldn't cut the would as a whole piece from the middle of the trunk like the texture does. However, it was a bit out of the scope of this project and my abilities to simulate real wood carving. In general this render proposes a believable wood texture, in which the main ask is a computed displacement for the texture, which brings us to the last texture.

Figure 2. Wood texture render with Stanford bunny and wooden planks, black background. Rendered until convergence in a 1080x1080 frame. See `./project/renders/wood_1024.png` for clearer version.

4.3 Rocky Lava

It an important note for this texture that it was originally developed on a Blender icosphere object (with 6 subdivisions), i.e. an object with a large number if small triangles that makes a sphere-like object. This is important for how the displacement in the texture is created. For that reason the special object for this texture is an icosphere. The results for this render can be seen in Figure 3. The first thing one might notice is, that the Stanford bunny now looks like a miserable burnt pigeon. That is due to the displacement and how it works on trimesh objects. While the icosphere is made of standard small triangles with identical properties, the trimesh object is not. Here the triangles are of different sizes and are shaped quite differently. This does not play well with the displacement logic. On a more smooth trimesh object, like the one shown in Figure 4 it works a lot better.

Apart from the misshaped bunny, there icosphere turned out quite well. The glow resembles what one (who admittedly has never seen real lava) might expect, with the colors somewhat matching those, which are created by high temperatures of lava. The rocky parts are sticking out with expected sharp edges, and the mixing of textures and displacements looks random and natural. Of course, lava usually doesn't clump into a ball like it is in the image, but mapped to a proper "river"-like object this could be believable.

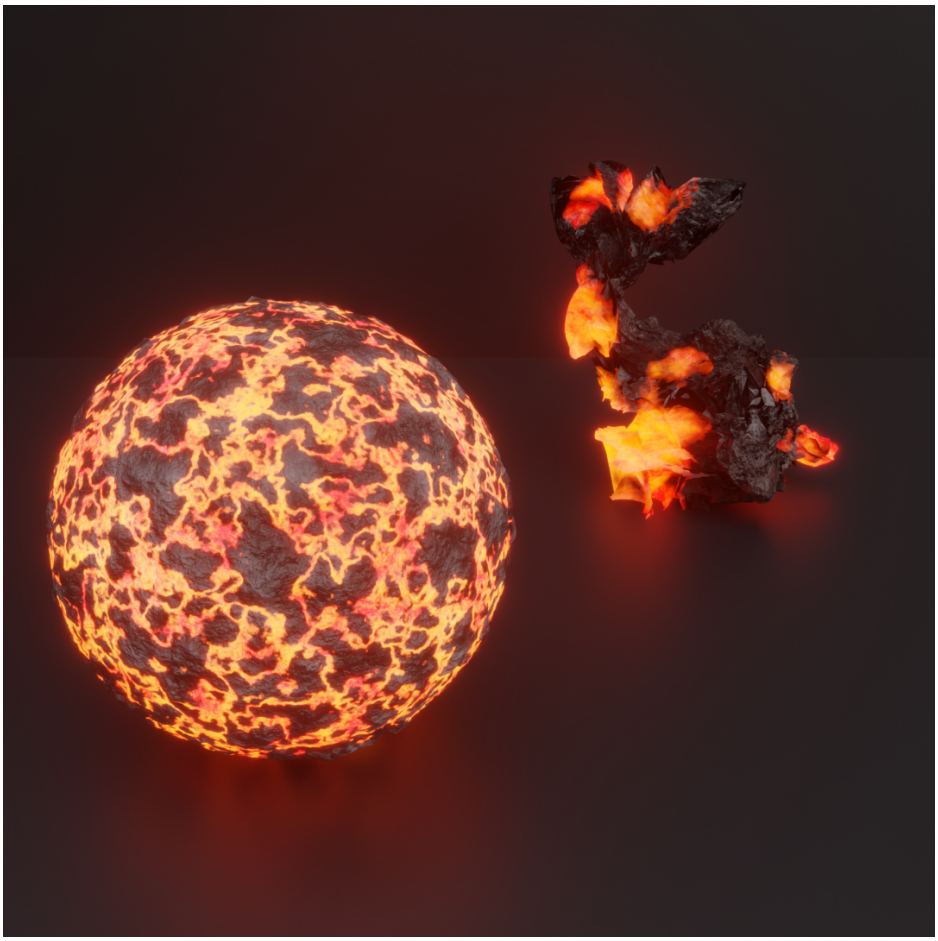

Figure 3. Rocky lava texture render with Stanford bunny and lava ball, black background. Rendered until convergence in a 1080x1080 frame. See `./project/renders/lava_sphere_1024.png` for clearer version.

5 DISCUSSION

Procedural texturing is not a perfect technique and it certainly comes with some downsides. One of the biggest downsides is the process of creating the texture. It is a manual process, and requires a fair amount of trial and error before reaching an acceptable product. Of course, with training, this process will become easier, but humans actively imitating nature will never yield a perfect result. This is seen as one of the main downfalls of the marble texture in Figure 1. It comes close to the real version of marble, but looking closely it is obvious that it is man made. We also see it in the wood texture (Figure 2) where the planks are too clean cut and straight.

Looking further at the rendered textures we notice some other shortcomings. The lava-textures bunny is an obvious one. It represents how certain textures with certain objects create unnatural looking renders due to the automatically generated surfaces. In the end, this is of course an avoidable problem, as we shouldn't design displaced textures for object with predisposed textures, like the bunny. It limits the displaced textures to objects with mostly convex shapes like the icosphere or the elephant.

Lastly, we should address what might be the elephant in the room (not the one in Figure 4). Adding complexity to the textures significantly increases the processing time of the render process. In this project the marble and wood textures took mere minutes to render, while the lava texture took almost an hour. While loading an image of lava might take more memory it would be faster to map that to the objects. This of course creates a bunch of other issues with realism, but it could be a helpful alternative in certain examples.

In conclusion, building textures with a tool like Blender is a powerful way of creating pseudo-realistic object surfaces. With relatively little experience and time one can make something that is believable and useful. On the other the other hand creating the level of hyper-realism often used in movies and games is a time consuming process and might end up being heavy on processing time.

REFERENCES

King, R. (2023). Procedural rocky lava material (blender tutorial). `https://www.youtube.com/watch?v=yg5RCNgV57M` Accessed: 2025-11-30.

Patricio Gonzalez Vivo, J. L. (2015). *The Book of Shaders*. Self-published.

wikipedia (2025). Perlin noise. `https://en.wikipedia.org/wiki/Perlin_noise` Accessed: 2025-11-30.

6 GENAI USAGE

In this project and the attached exercises GenAI has been used as an inspirational tool as well as an error searcher and debugger. For the project the main usage has been to understand nodes and features in Blender. No text or content has been generated with GenAI.

In the weekly exercises GenAI has been used to understand WGSL and javascript functions (including making comments for the code to increase understanding). Some lesser parts of the functional code has been generated using GenAI tools to help with inspiration and understanding of features and implementations. Additionally the style sheet (.css file) has been generated exclusively using GenAI.

A - EXTRA ROCKY LAVA TEXTURE RENDERINGS

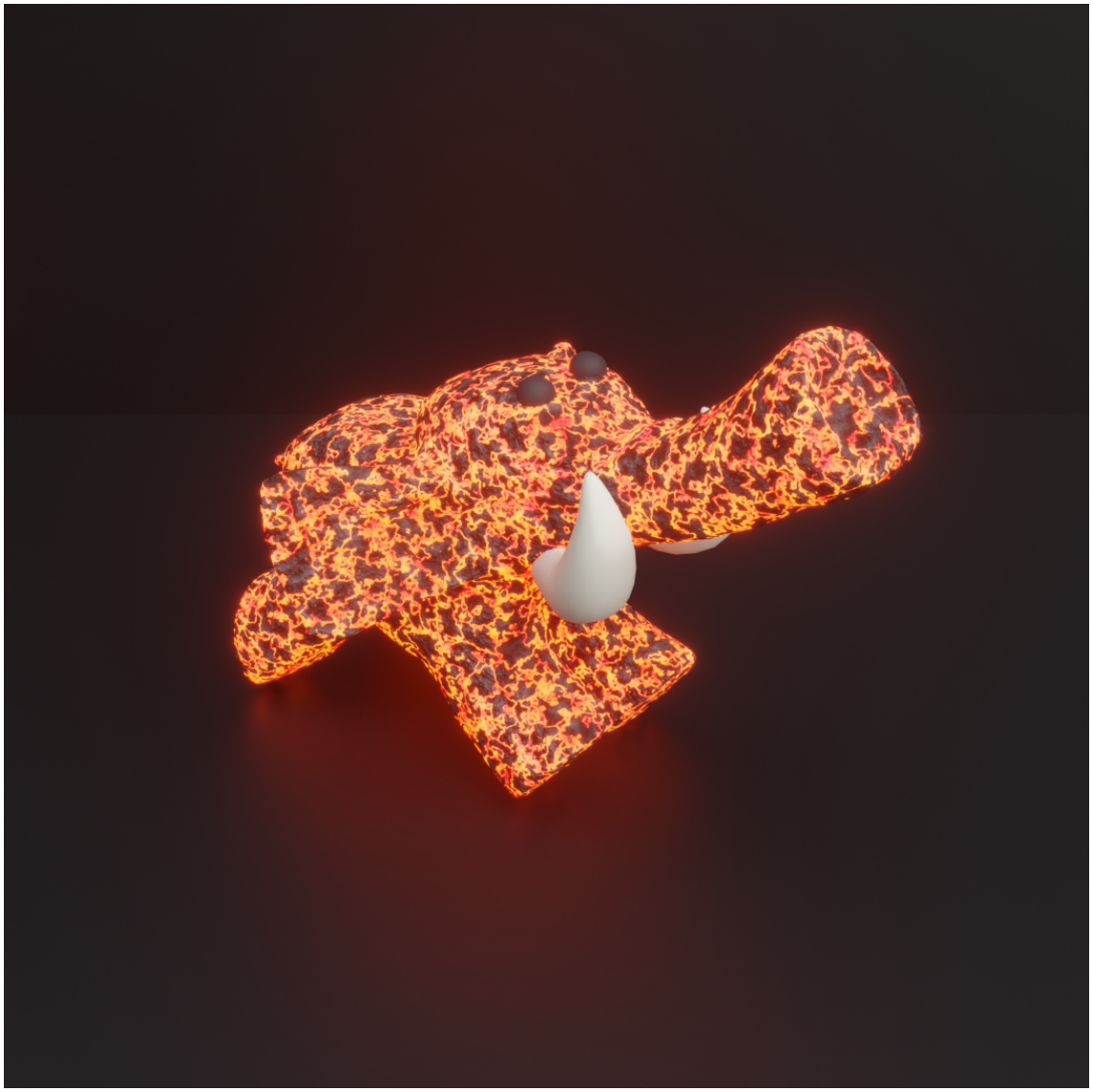

Figure 4. Rocky lava texture render with Elephant and black background. Rendered until convergence in a 1080x1080 frame. See `./project/renders/lava_elephant_1024.png` for clearer version.